



Universidad
Nacional
de Loja

Universidad Nacional de Loja

Facultad de la Energía, las Industrias y los Recursos Naturales
No Renovables

Carrera de Computación

Árboles de Recursión y Análisis de Complejidad de Algoritmos

Complejidad Computacional.

AUTOR:

Yandri Alexander Piscocama Jaramillo

DOCENTE:

Ing. Oscar Cumbicus

Loja, Ecuador

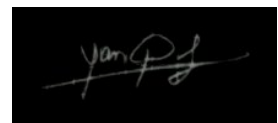
24 de mayo del 2026

Declaración de Uso de Inteligencia Artificial Generativa

En la elaboración del presente informe se utilizaron las siguientes herramientas de inteligencia artificial y repositorios digitales como apoyo al proceso investigativo:

- **Claude (Anthropic):** Asistente de inteligencia artificial empleado para la redacción, estructuración y análisis del contenido académico del informe, bajo supervisión y validación del autor.
- **Elicit:** Plataforma de búsqueda bibliográfica asistida por IA, utilizada para identificar y recuperar fuentes académicas relevantes sobre árboles de recursión y complejidad computacional en repositorios como ACM Digital Library e IEEE Xplore.
- **Mendeley:** Gestor de referencias bibliográficas empleado para organizar, almacenar y exportar las citas en formato BibTeX.
- **Google NotebookLM (Notebook):** Herramienta de organización y síntesis de información, utilizada para consolidar los bloques de contenido extraídos de las fuentes antes de la redacción final.

El autor declara que el uso de estas herramientas se realizó de manera transparente y responsable, que todo el contenido fue revisado y validado personalmente, y que la responsabilidad intelectual del trabajo corresponde íntegramente al firmante.



Yandri Alexander Piscocama Jaramillo

Loja, Ecuador, 24 de mayo del 2026

Índice General

Declaración de Uso de IA Generativa	1
Objetivo	4
I Parte I: Consulta Teórica	4
I.1 ¿Qué es la recursividad?	4
I.2 ¿Qué es un árbol de recursión?	4
I.3 Utilidad de los árboles de recursión en el análisis de algoritmos	5
I.4 Elementos de un árbol de recursión	5
I.4.1 Nodo raíz	5
I.4.2 Subproblemas	5
I.4.3 Altura del árbol	5
I.4.4 Coste por nivel	5
I.4.5 Coste total	6
I.5 Relación entre árboles de recursión y complejidad temporal	6
I.6 Ventajas y limitaciones del método de árboles de recursión	6
I.6.1 Ventajas	6
I.6.2 Limitaciones	6
II Parte II: Desarrollo Práctico	7
II.1 Ejercicio 1: $T(n) = 2T\left(\frac{n}{2}\right) + n$	7
II.1.1 Construcción del árbol	7
II.1.2 Coste por nivel	7
II.1.3 Altura del árbol	7
II.1.4 Coste total y complejidad	7
II.1.5 Árbol de recursión	8
II.2 Ejercicio 2: $T(n) = T(n - 1) + 1$	8
II.2.1 Construcción del árbol	8
II.2.2 Estructura de cadena lineal	8
II.2.3 Altura del árbol	8
II.2.4 Coste total y complejidad	9
II.2.5 Árbol de recursión	9
II.3 Ejercicio 3: Merge Sort	9

II.3.1	Problema que resuelve	9
II.3.2	Funcionamiento general	10
II.3.3	Árbol de recursión correspondiente	10
II.3.4	Complejidad computacional	10
II.3.5	Diagrama del proceso	11
III Parte III: Reflexión		12
Referencias Bibliográficas		13

Objetivo

Investigar y comprender el funcionamiento de los árboles de recursión como técnica para analizar algoritmos recursivos y determinar su complejidad computacional.

I. Parte I: Consulta Teórica

I.1. ¿Qué es la recursividad?

La recursividad es una metodología de diseño de algoritmos en la que un procedimiento se invoca a sí mismo para completar su ejecución, constituyendo una forma alternativa de repetir un proceso [1]. Su principio central es que *el todo puede representarse a través de sus partes*, lo cual implica reducir el tamaño del problema original hasta resolver subproblemas más pequeños que, posteriormente, se combinan para obtener la solución global [1]. A diferencia de la iteración —que emplea estructuras de bucle para acumular resultados de forma directa e intuitiva—, la recursividad requiere almacenar estados intermedios en la pila de llamadas (*call stack*), lo que supone un consumo adicional de memoria [1].

I.2. ¿Qué es un árbol de recursión?

Un árbol de recursión es una herramienta visual que permite estimar la solución de una recurrencia de manera intuitiva [2]. Se representa como un esquema en el que cada nodo corresponde al coste de un subproblema en un nivel específico de la ejecución: el nodo raíz contiene el problema original, y sus nodos hijos son las llamadas recursivas que este genera [2]. En esencia, constituye la traza de ejecución de un programa convertida en forma arbórea, lo que permite observar con claridad cómo se distribuye el trabajo computacional a lo largo de todas las llamadas del algoritmo [2].

I.3. Utilidad de los árboles de recursión en el análisis de algoritmos

Los árboles de recursión son fundamentales para visualizar cómo se distribuye el coste computacional entre los distintos niveles de llamadas [2]. Permiten derivar cotas de complejidad temporal tanto para algoritmos de tipo *divide y vencerás* como para recurrencias lineales [2]. A diferencia del Teorema Maestro —que es una fórmula directa aplicable a recurrencias de la forma $T(n) = aT(n/b) + f(n)$ — el árbol de recursión es un método más general, aplicable a una clase más amplia de recurrencias, y sirve como punto de partida para formular conjeturas que luego se verifican mediante inducción matemática [2].

I.4. Elementos de un árbol de recursión

I.4.1. Nodo raíz

Representa el problema inicial en el nivel superior ($i = 0$). Contiene el coste del trabajo realizado por la llamada original antes de dividirse en subproblemas [2].

I.4.2. Subproblemas

Son los nodos hijos generados por cada llamada recursiva. En cada nivel representan una fracción o reducción del tamaño de la entrada del nodo padre [2].

I.4.3. Altura del árbol

Es el número de niveles que tiene el árbol, es decir, cuántas veces puede reducirse la entrada antes de alcanzar el caso base donde la recursión se detiene. Por ejemplo, si el problema de tamaño n se divide a la mitad en cada paso, la altura es $\log_2 n$. Este valor también indica la profundidad máxima de la pila de llamadas [2].

I.4.4. Coste por nivel

Se obtiene sumando el trabajo realizado en todos los nodos que se encuentran a la misma profundidad i en el árbol [2].

I.4.5. Coste total

Es la suma de los costes de todos los niveles del árbol. Este valor representa la solución en forma cerrada que define la complejidad final del algoritmo [2].

I.5. Relación entre árboles de recursión y complejidad temporal

La estructura del árbol revela directamente el crecimiento del tiempo de ejecución. El análisis asintótico consiste en identificar si el coste total está dominado por el trabajo en la raíz, por el trabajo en las hojas, o si permanece constante en todos los niveles [2]. Concretamente, un algoritmo es $O(n \log n)$ cuando el coste en cada uno de los $\log n$ niveles es igual a n —como en Merge Sort—; por el contrario, es $O(n^2)$ cuando el árbol presenta una estructura en la que el trabajo decrece linealmente nivel a nivel, produciendo una suma aritmética de costes [1, 2]. En la computación de alto rendimiento contemporánea, se ha demostrado que el movimiento de datos entre niveles de memoria puede constituir un factor de coste tan relevante como el tiempo de cómputo [3], ampliando así la relevancia del análisis de complejidad más allá del tiempo de ejecución puro.

I.6. Ventajas y limitaciones del método de árboles de recursión

I.6.1. Ventajas

El método es altamente intuitivo y permite obtener clases de complejidad expresivas que incluyen términos logarítmicos, polinómicos y exponenciales. Es además tolerante a pequeñas imprecisiones en las trazas de ejecución, logrando obtener el resultado correcto de complejidad incluso cuando la información disponible es parcial [2].

I.6.2. Limitaciones

Resulta difícil de aplicar a recurrencias arbitrarias o muy complejas sin una conjetura previa. En algoritmos no deterministas —donde el tamaño de los subproblemas varía dinámicamente según condiciones de entrada, como en QuickSort con pivote aleatorio—

el árbol puede volverse inconsistente, dificultando un análisis preciso [2].

II. Parte II: Desarrollo Práctico

II.1. Ejercicio 1: $T(n) = 2T\left(\frac{n}{2}\right) + n$

II.1.1. Construcción del árbol

En el nivel 0 (raíz) existe un nodo con coste n . En el nivel 1 se generan 2 nodos, cada uno con coste $n/2$. En el nivel i existen 2^i nodos, cada uno con coste $n/2^i$ [2].

II.1.2. Coste por nivel

El coste acumulado en el nivel i es:

$$2^i \times \frac{n}{2^i} = n$$

El coste por nivel es constante e igual a n en todos los niveles del árbol [2].

II.1.3. Altura del árbol

Dado que el tamaño de la entrada se divide por 2 en cada nivel, la altura es $\log_2 n$ [2].

II.1.4. Coste total y complejidad

Sumando el coste constante n en cada uno de los $\log_2 n$ niveles:

$$T(n) = n \cdot \log_2 n \implies \boxed{O(n \log n)}$$

Esta recurrencia corresponde al comportamiento del algoritmo Merge Sort [1, 2].

II.1.5. Árbol de recursión

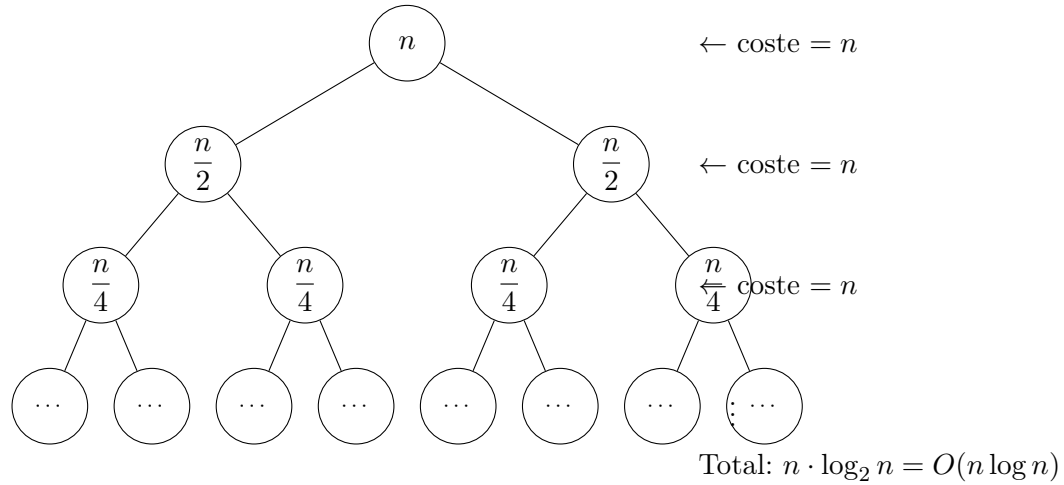


Figura II.1: Árbol de recursión para $T(n) = 2T(n/2) + n$. El coste por nivel es constante (n) en los $\log_2 n$ niveles, lo que produce una complejidad $O(n \log n)$.

II.2. Ejercicio 2: $T(n) = T(n - 1) + 1$

II.2.1. Construcción del árbol

En cada nivel i se genera un único nodo con coste constante 1 y una sola llamada recursiva a un problema de tamaño $n - 1$ [2].

II.2.2. Estructura de cadena lineal

El árbol adopta la forma de una cadena lineal porque el factor de ramificación es igual a 1: cada llamada genera exactamente una nueva llamada recursiva [2].

II.2.3. Altura del árbol

La altura es n , ya que la entrada se reduce en una unidad en cada nivel hasta alcanzar el caso base $T(0)$ [2].

II.2.4. Coste total y complejidad

Sumando el coste constante 1 en cada uno de los n niveles:

$$T(n) = \underbrace{1 + 1 + \cdots + 1}_{n \text{ veces}} = n \implies \boxed{O(n)}$$

II.2.5. Árbol de recursión

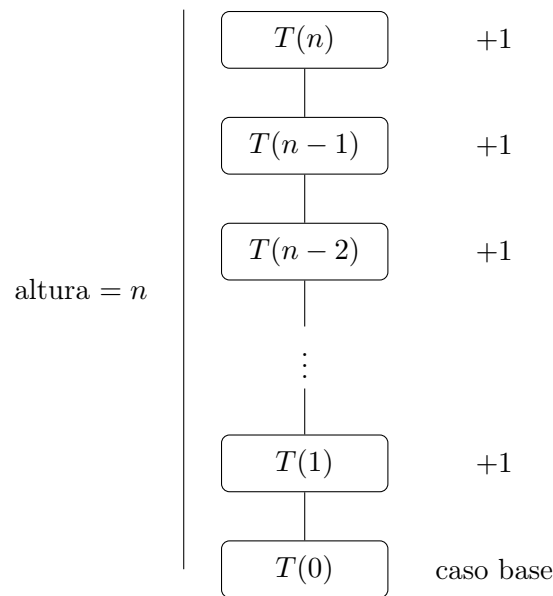


Figura II.2: Árbol de recursión para $T(n) = T(n-1) + 1$. Cadena lineal de altura n con coste unitario constante por nivel, produciendo complejidad $O(n)$.

II.3. Ejercicio 3: Merge Sort

II.3.1. Problema que resuelve

Merge Sort resuelve el problema de la ordenación de un conjunto de elementos, uno de los problemas fundamentales en las ciencias de la computación. La necesidad de ordenar datos aparece de forma constante como subproblema en áreas como la búsqueda eficiente, la gestión de bases de datos, el procesamiento de listas y la organización de estructuras de datos. Algoritmos de ordenación simples como Bubble Sort o Insertion Sort resultan ineficientes para entradas grandes al operar en tiempo $O(n^2)$, lo que los hace inviables en aplicaciones reales con volúmenes de datos elevados. Merge Sort surge como solución a esta limitación, garantizando una complejidad de $O(n \log n)$ en

todos los casos —mejor, promedio y peor— y preservando además la estabilidad del ordenamiento, es decir, el orden relativo de elementos con claves iguales [4].

II.3.2. Funcionamiento general

Opera bajo el paradigma *divide y vencerás*: primero divide el arreglo en dos mitades, ordena cada mitad de manera recursiva y, finalmente, ejecuta la función `merge` que fusiona ambas partes en una lista completamente ordenada [4].

II.3.3. Árbol de recursión correspondiente

Su árbol de recursión presenta una ramificación binaria constante en la que el trabajo de fusión en cada nivel es lineal (n). La recurrencia que describe su comportamiento es precisamente $T(n) = 2T(n/2) + n$, analizada en el Ejercicio 1, y el árbol tiene exactamente la misma estructura ilustrada en la Figura II.1 [2, 4].

II.3.4. Complejidad computacional

$$T(n) = O(n \log n)$$

Esta complejidad se obtiene sumando el trabajo de fusión n en cada uno de los $\log_2 n$ niveles del árbol [1, 4].

II.3.5. Diagrama del proceso

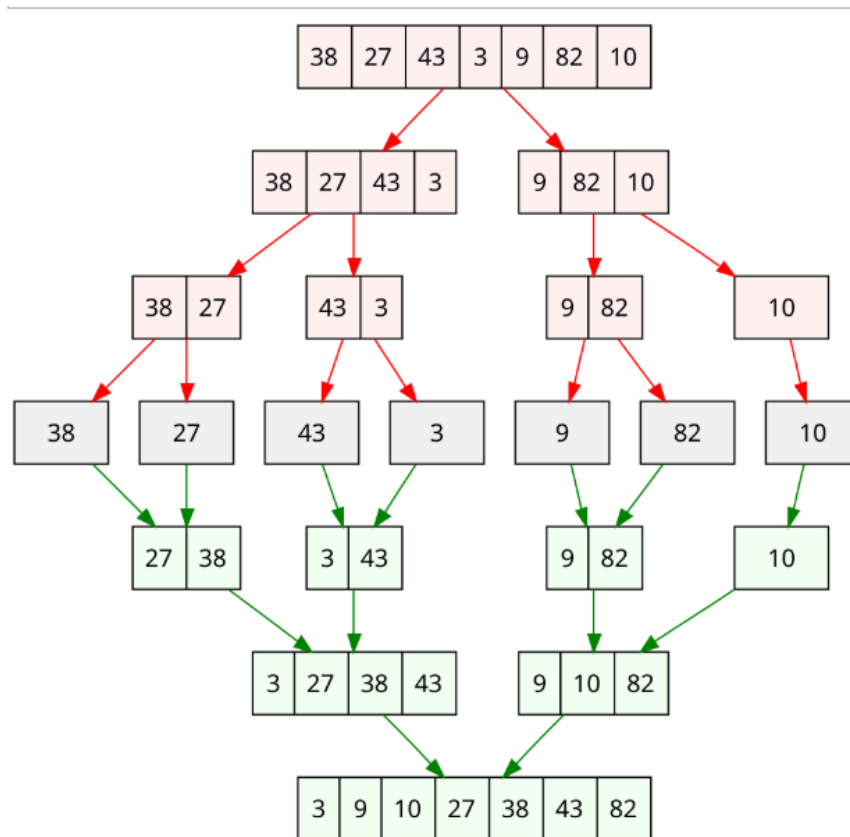


Figura II.3: Proceso de ordenación por Merge Sort bajo el paradigma divide y vencerás.

III. Parte III: Reflexión

El desarrollo de este trabajo me permitió comprender que analizar la complejidad de los algoritmos es una necesidad práctica antes que un ejercicio teórico: conocer si un algoritmo opera en tiempo $O(n \log n)$ o $O(n^2)$ determina directamente si un sistema puede escalar, responder en tiempo real o resistir condiciones de carga extrema. En la computación de alto rendimiento actual, incluso el movimiento de datos entre niveles de memoria representa un factor de coste tan relevante como el cómputo mismo, lo que confirma que este tipo de análisis seguirá siendo indispensable a medida que los sistemas crecen en escala y complejidad. Durante la construcción de los árboles de recursión, la mayor dificultad que encontré fue mantener la coherencia del diagrama cuando los subproblemas no siguen una reducción regular: en las recurrencias de este informe el proceso fue directo, pero comprendí que en algoritmos no deterministas el árbol puede volverse inconsistente y exigir el apoyo del Teorema Maestro o la inducción para validar la conjetura obtenida. Finalmente, estos análisis tienen aplicaciones vigentes en áreas de vanguardia: en modelos de lenguaje de gran escala, la descomposición recursiva adaptada a la complejidad del problema permite reducir el coste computacional sin sacrificar precisión, mientras que en genómica computacional las relaciones de recurrencia optimizan el alineamiento de secuencias de ADN sobre GPU con aceleraciones de varios órdenes de magnitud, lo que demuestra que los árboles de recursión no son un tema del pasado, sino una herramienta activa en la frontera del conocimiento computacional.

Referencias Bibliográficas

- [1] L. Chen, “Iteration vs. recursion: Two basic algorithm design methodologies,” University of the District of Columbia, Tech. Rep., 2005, lecture notes, Department of Computer Science.
- [2] D. Ishimwe, K. Nguyen, and T. Nguyen, “Dyaplex: Analyzing program complexity using dynamically inferred recurrence relations,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, Oct. 2021.
- [3] W. Smith, A. Goldfarb, and C. Ding, “Beyond time complexity: Data movement complexity analysis for matrix multiplication,” in *Proceedings of the International Conference on Supercomputing*. Association for Computing Machinery, Jun. 2022.
- [4] A. Aslam, M. S. Ansari, and S. Varshney, “Non-partitioning merge-sort: Performance enhancement by elimination of division in divide-and-conquer algorithm,” in *ACM International Conference Proceeding Series*. Association for Computing Machinery, Mar. 2016.